



# AMERICAN INTERNATIONAL UNIVERSITY–BANGLADESH (AIUB)

Dept. of Computer Science

Faculty of Science and Technology

## CSC2210: OBJECT ORIENTED PROGRAMMING 2

### Project Report

### Car Rental Management System

| Name                 | ID         |
|----------------------|------------|
| Jaydur Rahaman Shaon | 23-54136-3 |
| Fatiha Muhammad      | 23-55320-3 |
| Ashanur Jaman        | 23-51505-1 |
| Esrat Jahan          | 24-59371-3 |

GitHub Repository: <https://github.com/shaon4a-ui/CarRentalManagementSystem>

## Table of Contents

1. Title
2. Introduction
3. Case Study
4. ER Diagram
5. Normalization & Finalization
6. Database Schema
7. Database Query
8. Transition Table (Screen-by-Screen UI)

## Actual Application UI Screenshots

The following screenshots are taken directly from the implemented Car Rental Management System and show the actual user interface used during the customer authentication, dashboard, booking, and registration workflows.

### a) Login Screen

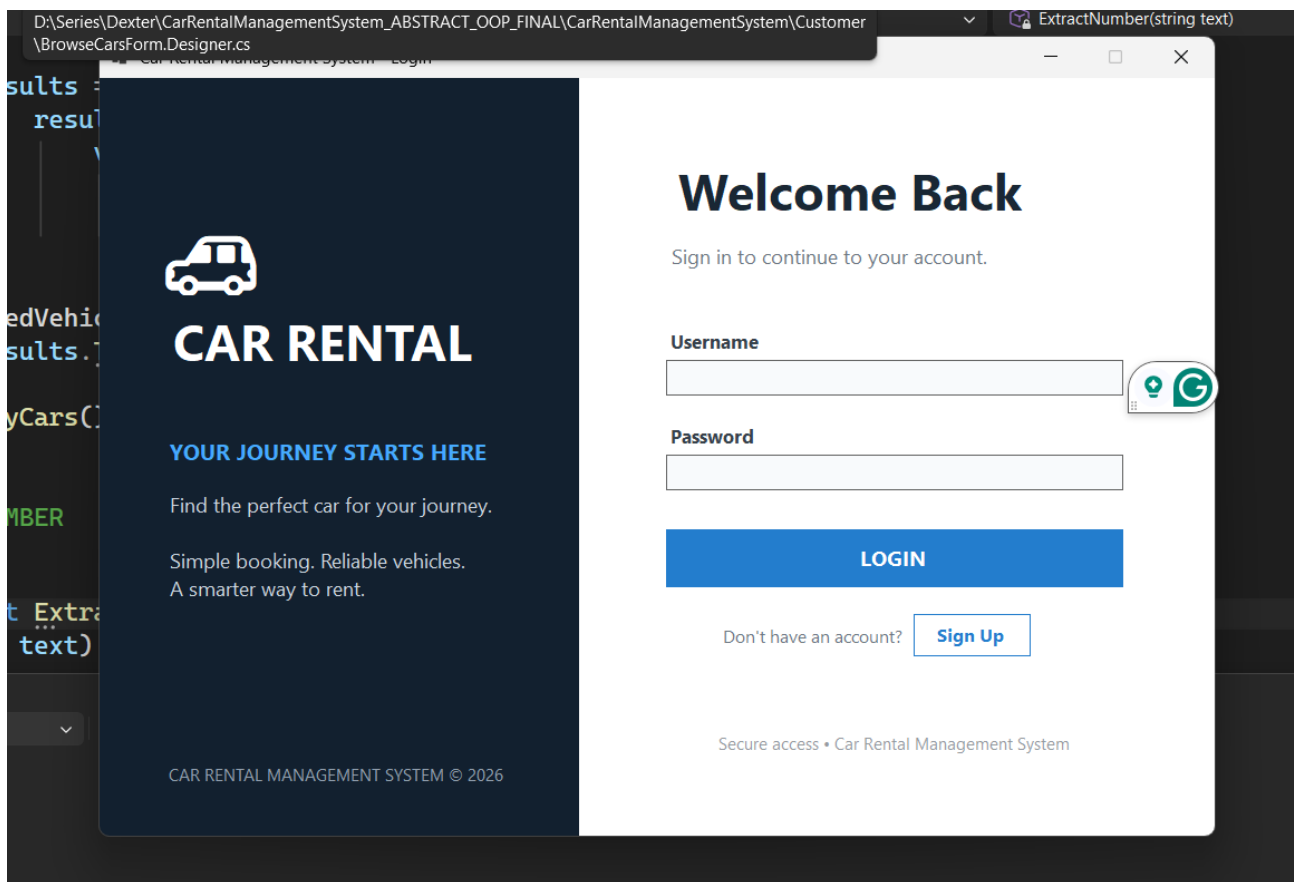


Figure: a) Login Screen

### b) Customer Dashboard

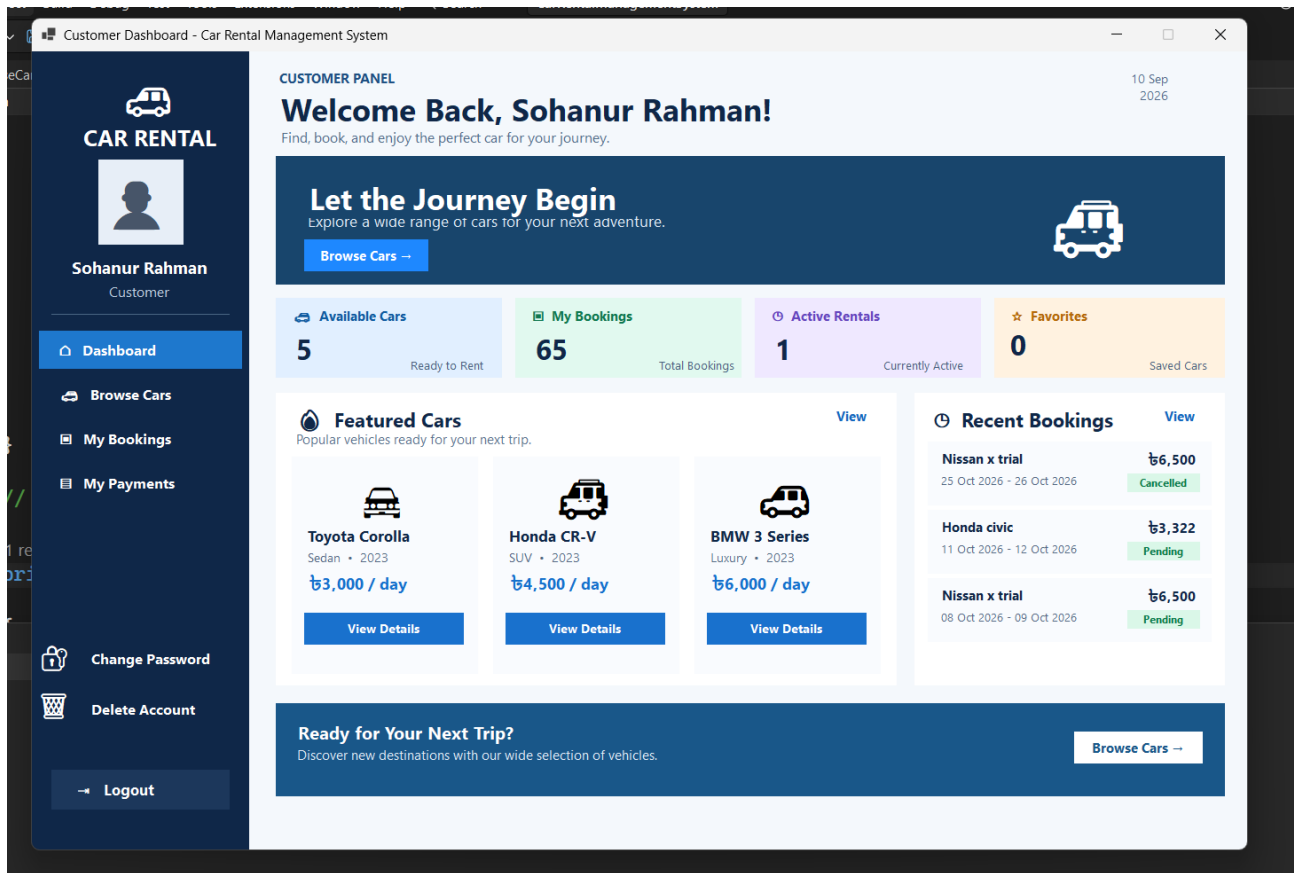


Figure: b) Customer Dashboard

### c) Book a Car – Available Date

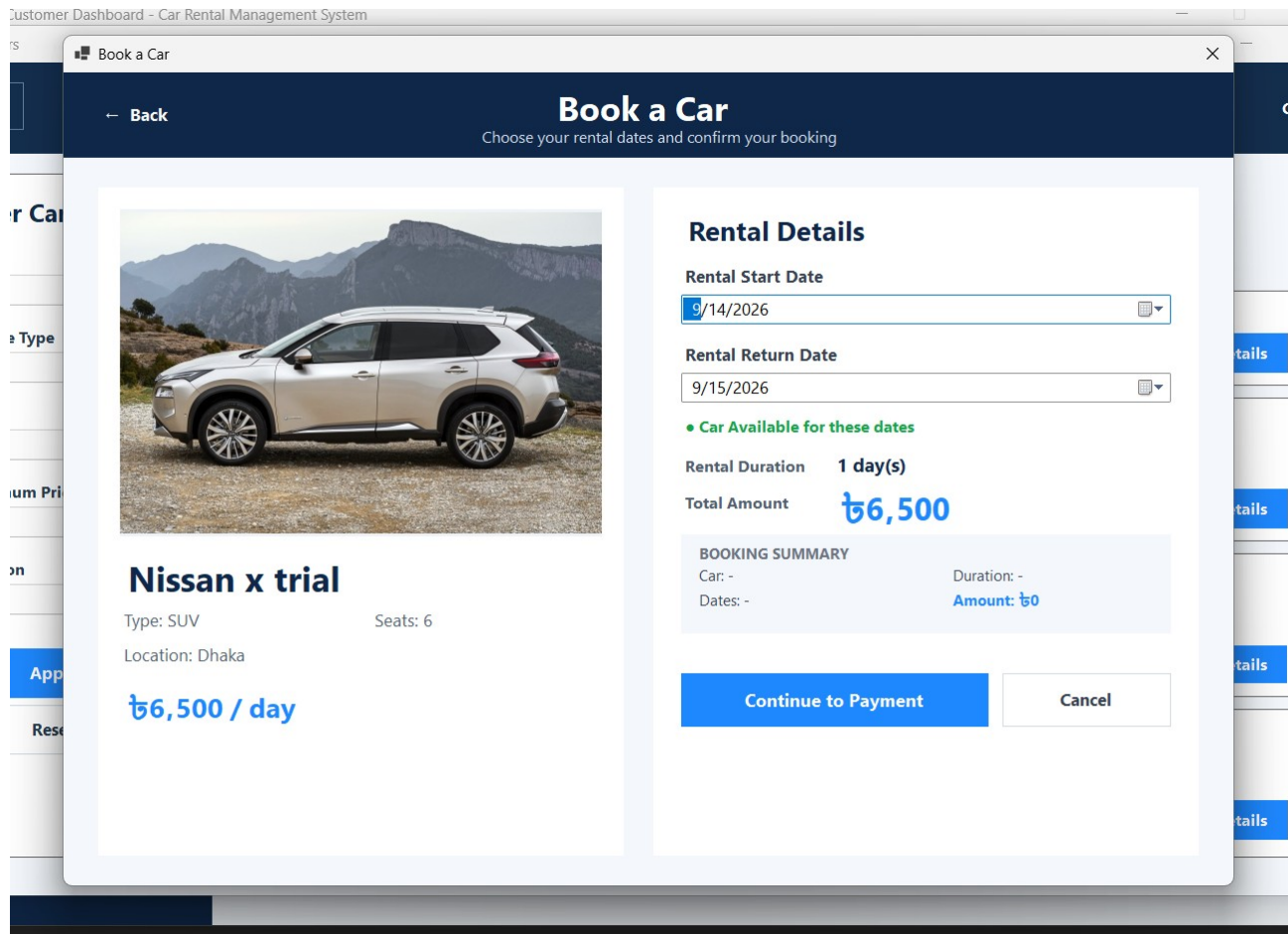


Figure: c) Book a Car – Available Date

**d) Book a Car – Unavailable Date**

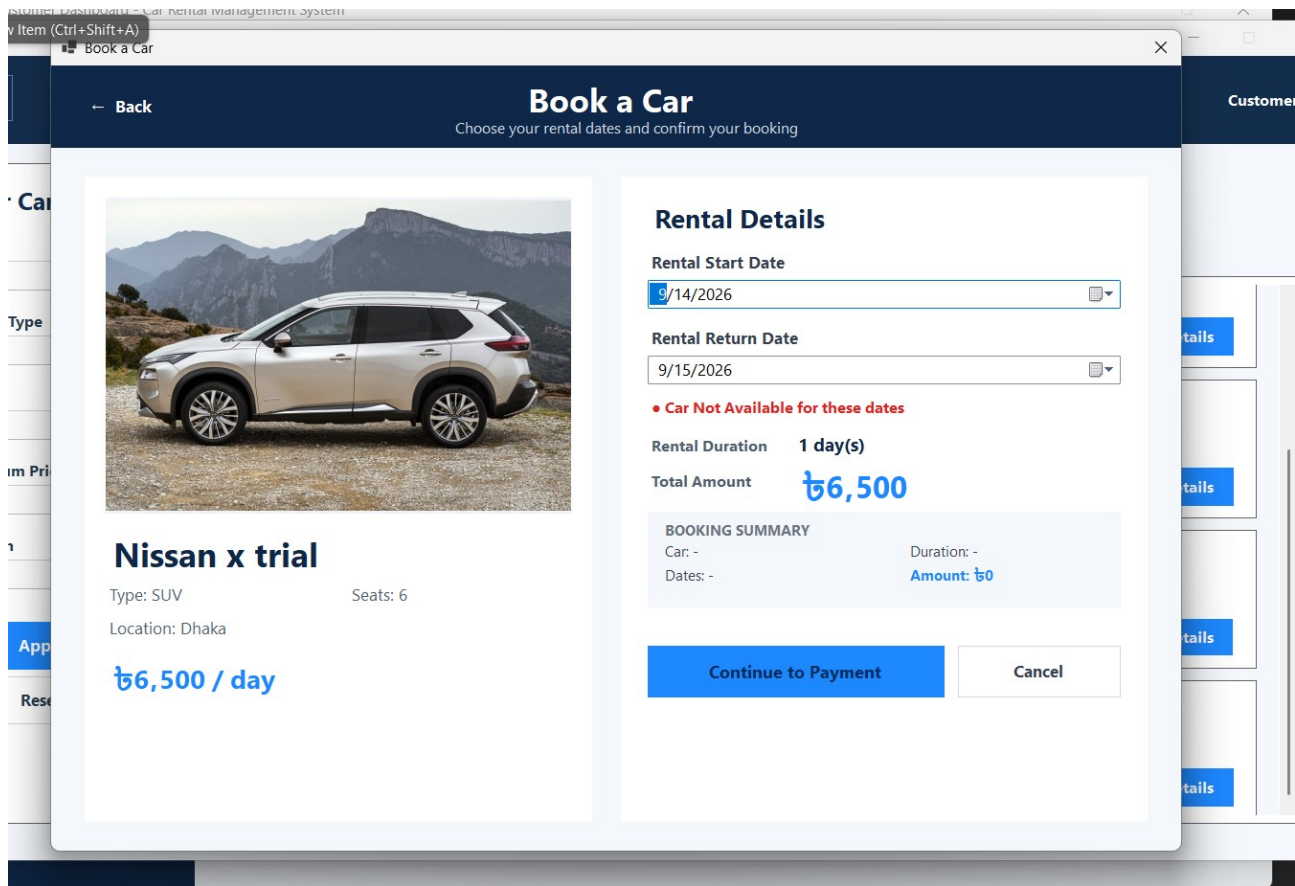


Figure: d) Book a Car – Unavailable Date

#### e) Create Account / Registration

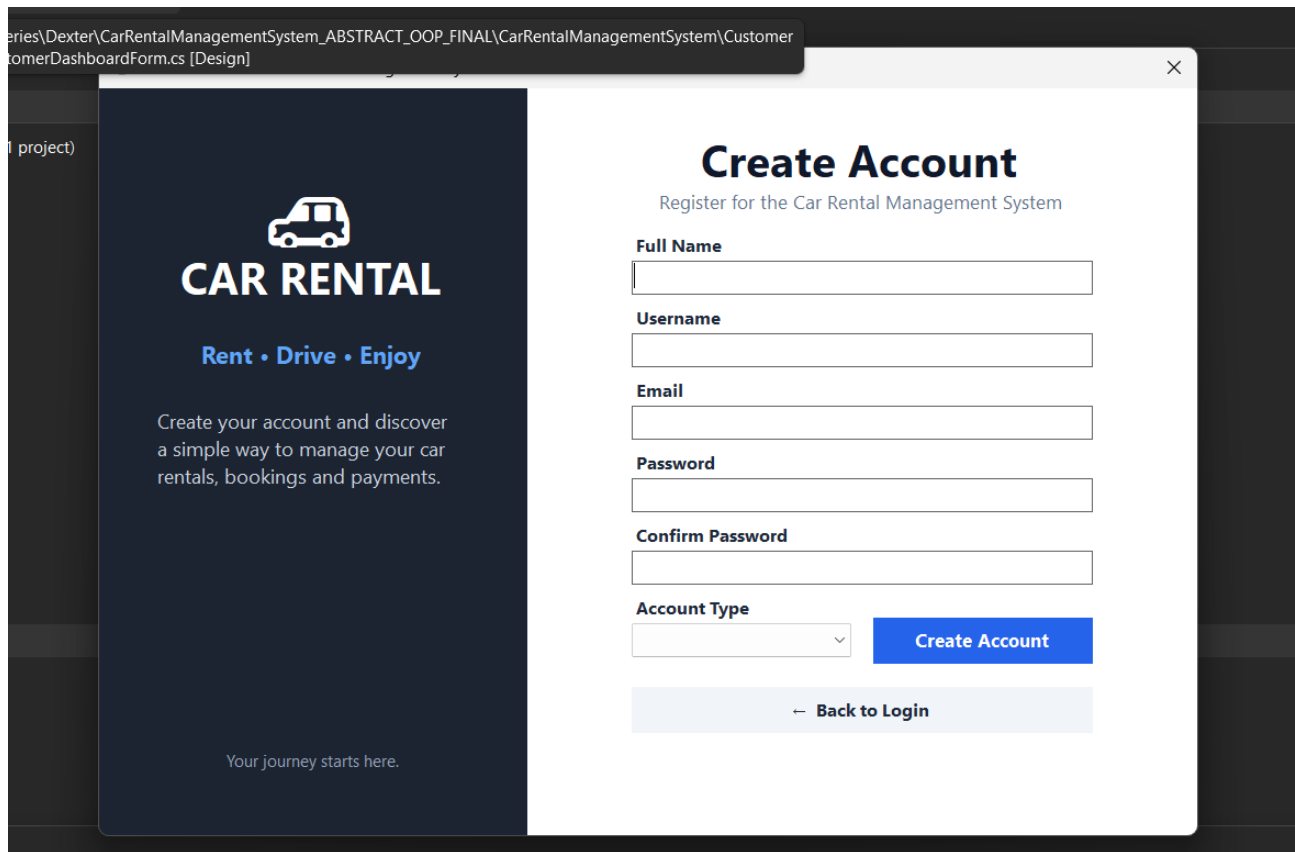


Figure: e) Create Account / Registration

## 9. Features and Facilities

## 10. Conclusion

# 1. Title

Car Rental Management System for streamlined vehicle rental, booking, payment, owner management, and administrative operations.

The project is a Windows desktop application developed with C# Windows Forms and SQL Server. It follows an object-oriented design with separate Customer, Owner, and Admin roles.

# 2. Introduction

A car rental business must coordinate vehicle availability, customer information, rental dates, bookings, payments, and owner operations. When these activities are handled manually, it becomes difficult to prevent overlapping reservations, maintain accurate booking records, and track payment and owner earnings. The Car Rental Management System addresses these activities through a role-based desktop application.

The application uses C# Windows Forms on .NET 10.0 for the graphical interface and SQL Server LocalDB for persistent data storage. Database communication is implemented through Microsoft.Data.SqlClient. Passwords are hashed using the ASP.NET Core Identity PasswordHasher rather than stored as plain text.

- Customer: browse available vehicles, search/filter cars, view details, create bookings, make payments, view bookings and payments, change password, and delete an eligible account.
- Owner: add and edit vehicles, manage owned cars, review bookings, confirm paid pending bookings, cancel unpaid pending bookings, and view earnings.
- Admin: manage users, all vehicles, and all bookings.
- Vehicle types are represented through Vehicle subclasses: Sedan, SUV, Van, and Luxury.
- Booking availability is checked using date-overlap logic so pending and confirmed bookings block conflicting rental intervals.

## Object-Oriented Design Used

| OOP Concept   | Implementation in the Project                                                                                             |
|---------------|---------------------------------------------------------------------------------------------------------------------------|
| Abstraction   | User and Vehicle are abstract base classes containing common data and behavior.                                           |
| Inheritance   | Customer, Owner, and Admin inherit from User; Sedan, SUV, Van, and Luxury inherit from Vehicle.                           |
| Encapsulation | Data is organized through class properties and form/service methods that operate on those objects.                        |
| Polymorphism  | The application loads vehicles as the common Vehicle type and instantiates the appropriate subclass based on VehicleType. |

### 3. Case Study

The system represents a real-life car rental business in which customers rent vehicles from registered owners while an administrator oversees the platform. The following case study follows the workflow implemented in the project source.

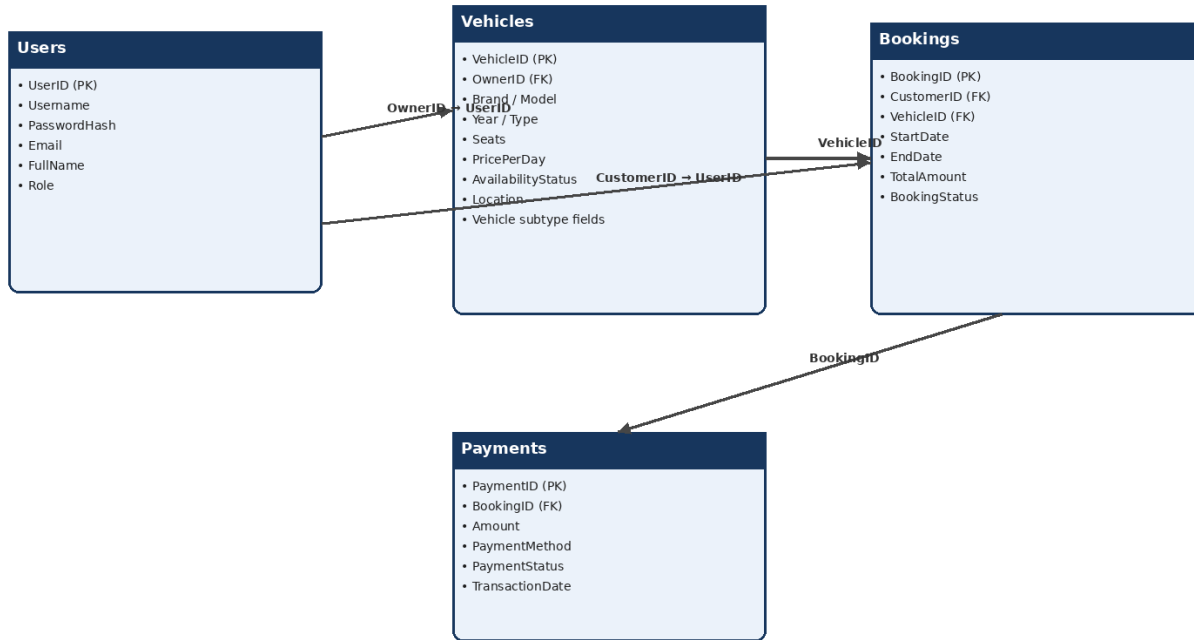
1. A user opens the application and reaches the Login screen. If the user does not have an account, the Sign Up screen allows registration as Customer or Owner.
2. After successful authentication, the system checks the Role stored in Users and opens the corresponding Customer, Owner, or Admin dashboard.
3. A customer opens Browse Cars. The system retrieves vehicles whose AvailabilityStatus is Available and displays vehicle cards with image, brand/model, type, seats, year, location, and price per day.
4. The customer may search/filter the available vehicles and open a vehicle details screen.
5. For a selected vehicle, the customer chooses a start date and return date. The system validates that the return date is after the start date and calculates the rental amount from the number of days multiplied by PricePerDay.
6. The system checks for overlapping Pending or Confirmed bookings using the standard interval condition: existing StartDate < requested EndDate and existing EndDate > requested StartDate.
7. If available, a Booking record is inserted with BookingStatus = Pending and the calculated TotalAmount.
8. The customer can then pay the exact booking amount. A successful payment is stored with PaymentStatus = Paid, while the booking remains Pending.
9. The owner reviews paid pending bookings and confirms the booking. Confirmed bookings contribute to owner earnings.
10. Unpaid pending bookings can be cancelled. Paid bookings are not cancelled because the project does not implement refunds.
11. The Admin dashboard provides system-wide management of users, cars, and bookings.

This lifecycle separates payment from owner confirmation. Therefore, a booking can be Paid while still Pending until the vehicle owner confirms it.

## 4. ER Diagram

The following logical ER diagram is derived from the project's C# models, database queries, and documented workflow. The four principal database entities are Users, Vehicles, Bookings, and Payments.

### Entity-Relationship Diagram - Car Rental Management System



Logical relationships derived from the project models, SQL queries, and application workflow.

Figure 1: Entity-Relationship Diagram of the Car Rental Management System

| Relationship        | Cardinality | Description                                                                               |
|---------------------|-------------|-------------------------------------------------------------------------------------------|
| Users → Vehicles    | 1 : Many    | An Owner user can own multiple vehicles.                                                  |
| Users → Bookings    | 1 : Many    | A Customer user can create multiple bookings.                                             |
| Vehicles → Bookings | 1 : Many    | A vehicle can have multiple bookings over time, provided rental intervals do not overlap. |
| Bookings → Payments | 1 : 0..1    | A booking can have one payment record; a unique index is recommended for BookingID.       |

## 5. Normalization & Finalization

The application uses four core relational tables: Users, Vehicles, Bookings, and Payments. The normalization discussion below describes how the main attributes can be separated according to their functional dependencies.

### Users

UNF: A combined user record may contain UserID, Username, PasswordHash, Email, FullName, Role, and CreatedAt.

1NF: Each user attribute contains a single atomic value and each user is identified by UserID.

2NF: Because the primary key is a single attribute (UserID), there are no partial dependencies.

3NF: Non-key user attributes depend on UserID rather than on another non-key attribute. Username is additionally treated as a unique login identifier.

### Vehicles

UNF: Vehicle information includes ownership, identity, classification, rental price, availability, descriptive information, location, image path, and subtype-specific fields.

1NF: Each field stores one value per vehicle record. The VehicleID uniquely identifies the vehicle.

2NF: With a single-column primary key VehicleID, there are no partial dependencies.

3NF: General vehicle attributes depend on VehicleID. OwnerID is a foreign key to Users, while VehicleType determines which subtype-specific properties are relevant. The project implements the vehicle hierarchy as a single Vehicles table with optional subtype fields, rather than separate subtype tables.

### Bookings

UNF: Booking information includes BookingID, CustomerID, VehicleID, StartDate, EndDate, TotalAmount, and BookingStatus.

1NF: All booking attributes are atomic and BookingID identifies one booking.

2NF: There are no partial dependencies because BookingID is a single-column key.

3NF: CustomerID and VehicleID are foreign keys; dates, amount, and status describe the booking identified by BookingID.

### Payments

UNF: Payment information includes PaymentID, BookingID, Amount, PaymentMethod, PaymentStatus, and TransactionDate.

1NF: All payment fields are atomic.

2NF: There are no partial dependencies because PaymentID is the single primary identifier.

3NF: Payment attributes depend on PaymentID, while BookingID links the payment to its booking.

### Final Tables

| Table    | Key attributes / important fields                                                                                                                       |
|----------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| Users    | UserID (PK), Username, PasswordHash, Email, FullName, Role, CreatedAt (where present in the deployed schema)                                            |
| Vehicles | VehicleID (PK), OwnerID (FK), Brand, Model, Year, VehicleType, Seats, PricePerDay, AvailabilityStatus, Description, ImagePath, Location, subtype fields |

| Table    | Key attributes / important fields                                                               |
|----------|-------------------------------------------------------------------------------------------------|
| Bookings | BookingID (PK), CustomerID (FK), VehicleID (FK), StartDate, EndDate, TotalAmount, BookingStatus |
| Payments | PaymentID (PK), BookingID (FK), Amount, PaymentMethod, PaymentStatus, TransactionDate           |

Note: The supplied project package contains the application code and a recommended integrity script rather than a complete CREATE TABLE database script. Therefore, the final table descriptions above are reconstructed from the actual model classes, SQL statements, connection configuration, and project workflow.

## 6. Database Schema

The application connects to the CarRentalDB database on SQL Server LocalDB using the configured connection string: Server=(localdb)\MSSQLLocalDB; Database=CarRentalDB; Trusted\_Connection=True; TrustServerCertificate=True.

### Relational Database Schema - Car Rental Management System

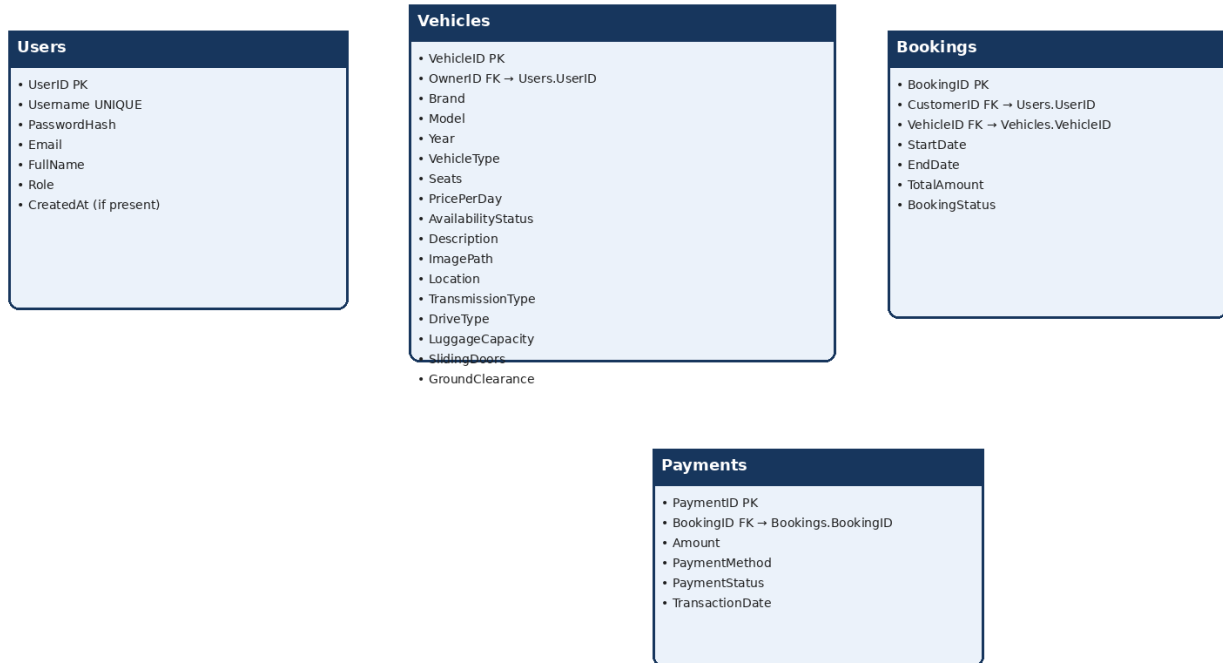


Figure 2: Relational Database Schema of the Car Rental Management System

The recommended database script supplied with the project adds integrity constraints such as  $EndDate > StartDate$ ,  $PricePerDay > 0$ ,  $Seats > 0$ , allowed BookingStatus values, allowed PaymentStatus values, and a unique index on Payments.BookingID. It also includes the subtype fields required by the C# vehicle classes.

## 7. Database Query

The application performs CRUD operations using parameterized SQL commands through Microsoft.Data.SqlClient. Representative queries from the source code are shown below.

### User Login Query

```
SELECT UserID, Username, PasswordHash, FullName, Role
FROM Users
WHERE Username = @Username;
```

### Create Booking Query

```
INSERT INTO Bookings
(
    CustomerID, VehicleID, StartDate, EndDate,
    TotalAmount, BookingStatus
)
OUTPUT INSERTED.BookingID
VALUES
(
    @CustomerID, @VehicleID, @StartDate, @EndDate,
    @TotalAmount, @BookingStatus
);
```

### Vehicle Availability Query

```
SELECT COUNT(*)
FROM Bookings
WHERE VehicleID = @VehicleID
AND BookingStatus IN ('Pending', 'Confirmed')
AND StartDate < @EndDate
AND EndDate > @StartDate
AND (@ExcludeBookingID = 0 OR BookingID <> @ExcludeBookingID);
```

### Payment Insert Query

```
INSERT INTO Payments
(
    BookingID, Amount, PaymentMethod, PaymentStatus
)
OUTPUT INSERTED.PaymentID
VALUES
(
    @BookingID, @Amount, @PaymentMethod, @PaymentStatus
);
```

### Owner Earnings Query

```
SELECT ISNULL(SUM(p.Amount), 0)
FROM Payments p
INNER JOIN Bookings b ON p.BookingID = b.BookingID
INNER JOIN Vehicles v ON b.VehicleID = v.VehicleID
WHERE v.OwnerID = @OwnerID
AND p.PaymentStatus = 'Paid'
AND b.BookingStatus = 'Confirmed';
```

## Database Integrity Queries

- A unique index on Payments.BookingID prevents duplicate payment records for the same booking.
- CHECK constraints validate booking date ranges, vehicle price, seat count, booking status, and payment status.
- The booking workflow uses parameterized commands and database transactions for payment insertion.

## 8. Transition Table (Screen-by-Screen UI)

The source project contains separate Windows Forms for authentication, customer operations, owner operations, and administrator operations. The transition below summarizes the screen sequence implemented in the code.

| Step | Screen                      | Main Action                                                   | Next Screen / Result                                      |
|------|-----------------------------|---------------------------------------------------------------|-----------------------------------------------------------|
| 1    | LoginForm                   | Enter username/password and authenticate                      | Role-based dashboard                                      |
| 2    | SignUpForm                  | Create Customer or Owner account                              | Return to Login                                           |
| 3    | CustomerDashboardForm       | Choose customer functions                                     | Browse Cars / My Bookings / My Payments / Account actions |
| 4    | BrowseCarsForm              | Search/filter and select an available vehicle                 | Car Details                                               |
| 5    | CarDetailsForm              | Review vehicle information                                    | Booking Form                                              |
| 6    | BookingForm                 | Choose rental dates; availability and total are calculated    | Payment Form or pending booking saved                     |
| 7    | PaymentForm                 | Select payment method and complete payment                    | Paid payment; booking remains Pending                     |
| 8    | MyBookingsForm              | View booking/payment status; manage eligible pending bookings | Updated booking status                                    |
| 9    | OwnerDashboardForm          | Open owner management functions                               | My Cars / Owner Bookings / Earnings                       |
| 10   | MyCarsForm / AddEditCarForm | Add or edit owned vehicles                                    | Updated vehicle list                                      |
| 11   | OwnerBookingsForm           | Review and confirm paid pending bookings                      | Confirmed booking                                         |
| 12   | EarningsForm                | View total/monthly earnings and completed bookings            | Earnings summary/chart                                    |
| 13   | AdminDashboardForm          | Open administrative modules                                   | Users / Cars / Bookings                                   |
| 14   | ManageUsersForm             | Search, add, edit, or delete eligible users                   | Updated user list                                         |
| 15   | ManageAllCarsForm           | View, edit, or delete eligible vehicles                       | Updated vehicle list                                      |
| 16   | ManageAllBookingsForm       | View and manage all bookings                                  | Updated booking status                                    |

## UI/UX Navigation Flow - Car Rental Management System

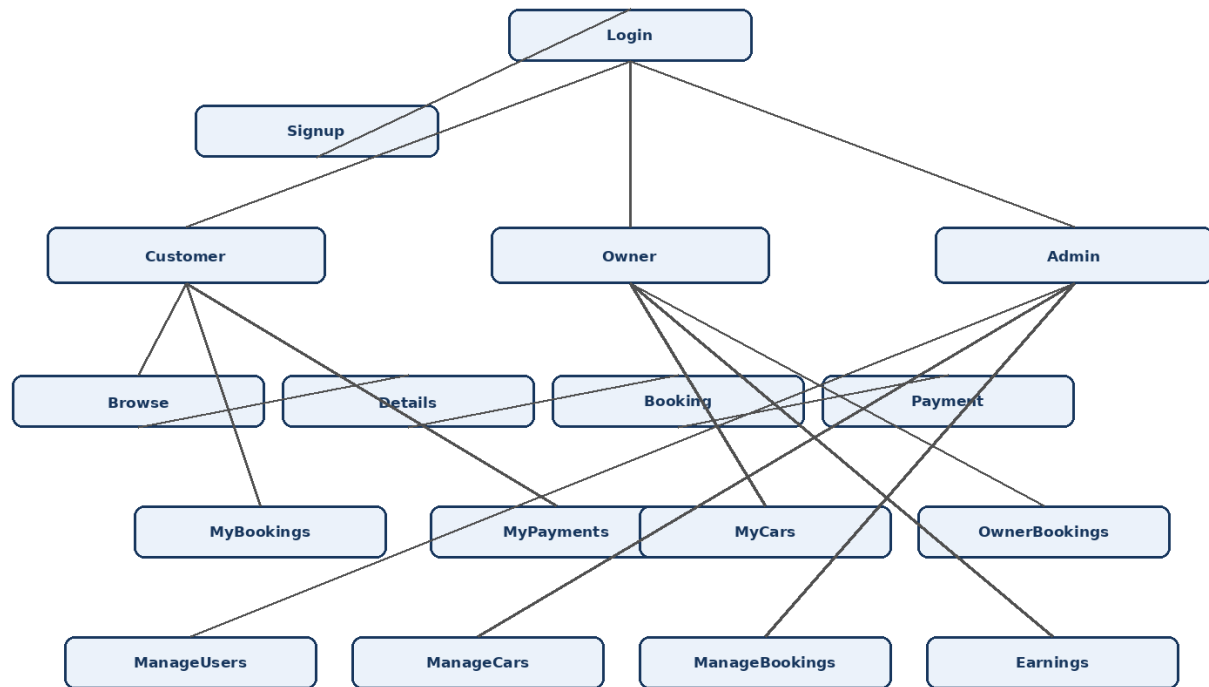


Figure 3: UI/UX Navigation Flow of the Car Rental Management System

The supplied ZIP contains the Windows Forms source and designer files but does not contain a standalone screenshot set. For the final submission, application screenshots can be inserted beside the corresponding rows above if required by the instructor.

## 9. Features and Facilities

- Role-Based Access: Customer, Owner, and Admin users are routed to separate dashboards and functions.
- Secure Authentication: Login verifies the stored password hash, while registration hashes passwords before insertion into Users.
- Vehicle Management: Owners can add and edit their vehicles; administrators can manage all vehicles.
- Vehicle Categories through OOP: Sedan, SUV, Van, and Luxury are represented as subclasses of the abstract Vehicle class.
- Vehicle Search and Filtering: Customers can search/filter by brand, vehicle type, seats, maximum price, and location through the Browse Cars interface.
- Vehicle Details and Images: Vehicle cards and details screens display vehicle information and load local vehicle images when available.
- Date-Based Availability: Booking availability uses date-overlap checking rather than simply relying on a permanent Available/Unavailable flag.
- Booking Lifecycle: New bookings are Pending, paid bookings remain Pending until owner confirmation, and unpaid pending bookings may be cancelled.
- Payment Processing: Payments record booking ID, amount, method, status, and transaction date; the payment operation uses a SQL transaction.
- Owner Earnings: EarningsService calculates total earnings, current-month earnings, pending payment value, confirmed booking count, and monthly earnings.
- Admin Management: Administrators can view and manage users, vehicles, and bookings, with safeguards against deleting records that are still related to bookings or vehicles.
- Validation: Required fields, password length, password confirmation, email format, rental dates, payment fields, and other input conditions are checked before database operations.
- Database Integration: The application uses SQL Server LocalDB and Microsoft.Data.SqlClient for persistent storage and retrieval.
- User-Friendly Windows Forms UI: The project uses dashboard navigation, buttons, labels, text boxes, combo boxes, DataGridView controls, vehicle cards, and validation messages.

### Security and Data Integrity

- Passwords are stored as hashes using PasswordHasher rather than plain text.
- SQL commands use parameters such as @Username, @OwnerID, @BookingID, and @VehicleID.
- Payment insertion is wrapped in a SQL transaction.
- Booking confirmation is allowed only when a corresponding Paid payment exists.
- Paid bookings are protected from the unpaid-booking cancellation path.
- User and vehicle deletion checks prevent deletion when related bookings/ownership records exist.

## 10. Conclusion

The Car Rental Management System provides a practical desktop solution for managing vehicle rentals through a structured, role-based workflow. Customers can discover vehicles, select rental dates, make bookings, and complete payments; owners can manage vehicles, review bookings, confirm paid reservations, and monitor earnings; administrators can supervise users, vehicles, and bookings.

From an Object-Oriented Programming perspective, the project demonstrates abstraction, inheritance, encapsulation, and polymorphism through its User and Vehicle class hierarchies. From a database perspective, it integrates SQL Server with relational entities for users, vehicles, bookings, and payments, supported by validation, foreign-key relationships, date-overlap checking, and recommended integrity constraints.

Overall, the system demonstrates how C# Windows Forms, object-oriented design, SQL Server database integration, authentication, booking logic, payment processing, and role-based access can be combined to model a real-life car rental operation in a single desktop application.